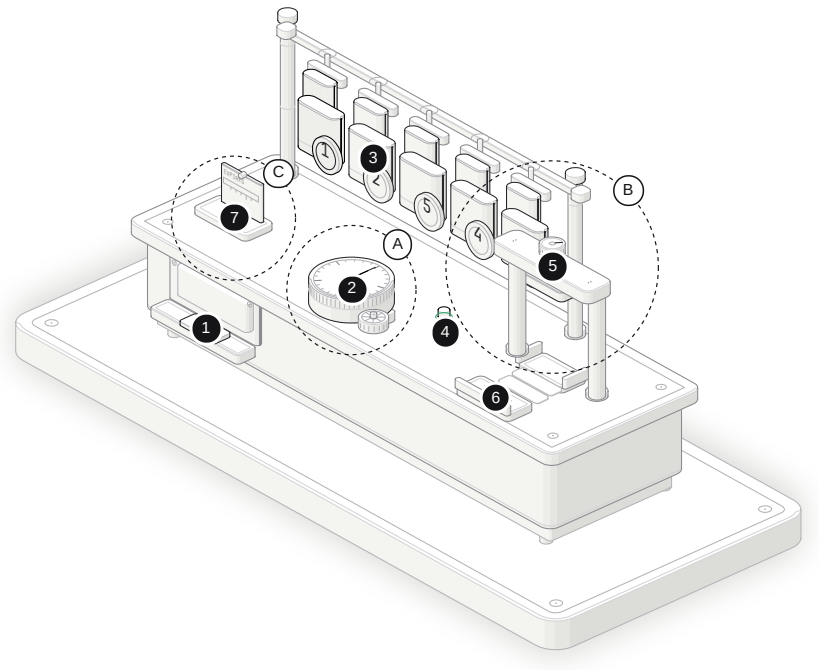
304



How does a response stay fresh without a cache to go stale?

Freshness is left to HTTP itself. A coat's tag is its ETag — a checksum of Modified and Length — so a ticket that still matches gets the very same ticket back as 304 Not Modified and the coat stays on the rail. A stale ticket brings the coat down and the press hands it out folded — brotli, gzip or zstd, whichever Accept-Encoding admits.

0 copies
NO SERVER-SIDE CONTENT CACHE, BY DESIGN

2 fields
IN THE ETAG CHECKSUM: MODIFIED + LENGTH

3 codecs
BROTLI, GZIP, ZSTD AGAINST ACCEPT-ENCODING

304 on match
SAME TICKET BACK, COAT STAYS ON THE RAIL

1 stat
TO PROVE CONTENT UNCHANGED

PARTS

01 **The claim ticket**
If-None-Match, handed in at the counter.

02 **The comparator**
Two dials compared: the ticket against the coat's tag.

03 **A coat on the rail**
The representation, its tag the ETag — checksum of Modified + Length.

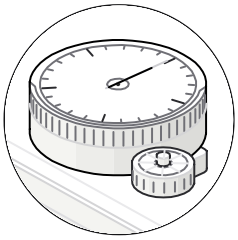
04 **The green lamp**
Tag matches: 304 Not Modified, no coat comes down.

05 **The press**
Brotli, gzip or zstd — the coat folded to what the client admits.

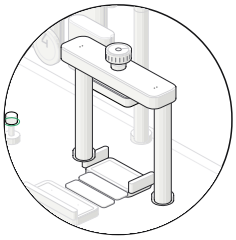
06 **The out tray**
The folded bundle handed over the counter.

07 **The Expires card**
How long a client may trust what it holds.

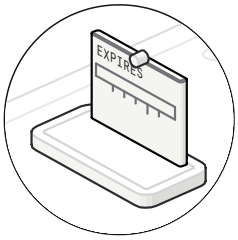
CLOSE-UPS



(A) **The comparator**
CacheValidationHandler compares the computed checksum with If-None-Match — and documents both headers through IDocumentedConcern.

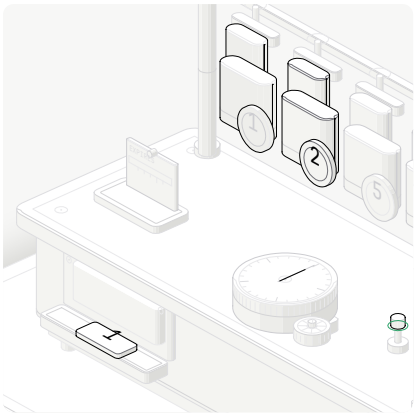


(B) **The press**
CompressedContent rebuilds through IResponse.Rebuild(): brotli, gzip and zstd, chosen against Accept-Encoding.



(C) **The card**
Expires is stamped as RFC 1123, for GET and 200 responses, gated by an optional predicate.

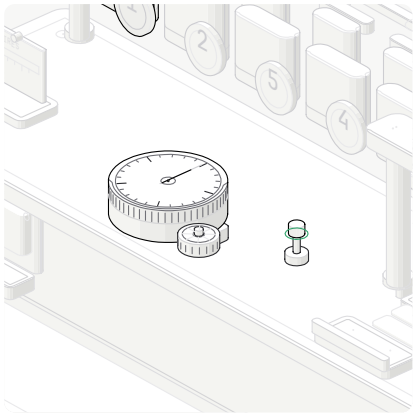
How it works



01

No spare copies

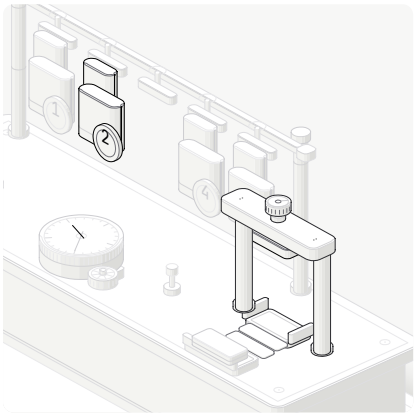
The coat hangs on the rail and the cloakroom keeps nothing else. Its tag is an ETag checksummed from Modified and Length.



02

Match: the same ticket

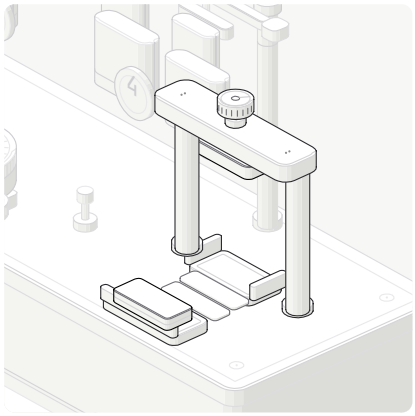
If-None-Match still matches, so the response is rewritten to 304 Not Modified in place — the coat never leaves the rail.



03

Stale: the coat comes down

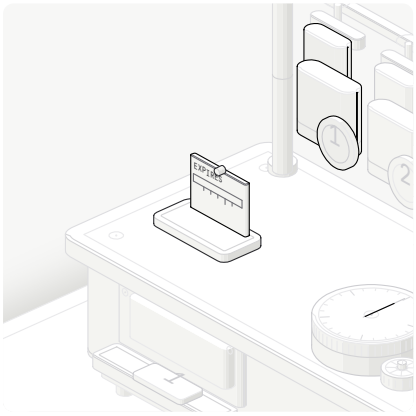
A stale ticket re-wraps: the representation is rebuilt — same pooled response, new compressed content.



04

Folded to size

Only compressible types above a minimum size are pressed; already-compressed content is skipped, and .br siblings are served straight away.



05

Trust, with a date

ClientCache.Policy() adds Expires to GET/200 responses — how long the client may trust its coat before asking again.

WHY NO CACHE?

A server-side cache can serve stale bytes; a validator cannot. The ETag derives from the content's identity — Modified plus Length — so revalidation checks the truth itself, and a 304 costs only the comparison. Size is the client's call: compression respects Accept-Encoding, and nothing is folded that was already folded.

SPECIFICATION

Validator	ETag from CalculateChecksumAsync	Rebuild	same pooled response, new wrapper
Match	If-None-Match rewrites to 304	Pre-compressed	.br / .gzip siblings served directly
Policy	ClientCache.Policy() adds Expires	Ranges	RangeSupport through a RangedSink
Compression	CompressedContent.Default()		